Posts and Telecommunications Institute of Technology

Faculty of Information Technology 1

Software Architecture and Design Essay

Building an E-Commerce System Using Microservices and AI

Lecturer: Tran Dinh Que
Student: B22DCCN441 - Nguyen Duc Khai
Class: E22CNPM01
Date: 2026

Contents

List of Figures

List of Tables

1 From Monolithic Architecture to Microservices and DDD

1.1 Introduction to Monolithic Architecture

1.1.1 Concept

Monolithic Architecture is an architectural style in which the presentation layer, business logic, and data access layer are packaged and deployed as a single application. In a simple e-commerce system, user management, product catalog, cart, order, payment, shipping, and recommendation logic may all be located in the same Django project and connected to the same database.

1.1.2 Typical Structure

A typical monolithic application contains three internal layers:

- **Presentation layer**: web pages, forms, templates, or frontend screens.
- **Business logic layer**: product rules, order calculation, payment flow, shipping status, and recommendation logic.
- **Data access layer**: database models, queries, repositories, and migrations.

The layers may be internally organized, but they are still deployed as one runtime unit.

1.1.3 Practical E-Commerce Example

For an e-commerce system, a monolithic Django application may include:

- Product and category management.
- User registration, authentication, and role management.
- Cart and checkout.
- Order history.
- Payment and shipping status.
- Product recommendation and chatbot modules.

This is convenient for a small MVP because development and deployment are simple. However, the architecture becomes difficult to maintain as the domain grows.

1.1.4 Detailed Drawbacks

- **Hard to scale**: the whole application must be scaled even if only catalog browsing is under heavy load.
- **High coupling**: a change in payment logic can affect unrelated product or order modules.
- **Risky deployment**: one small bug can break the whole system.
- **Difficult team development**: many developers modify the same codebase and database schema.
- **Weak domain boundaries**: technical folders may hide business dependencies.

1.1.5 When to Use Monolithic Architecture

Monolithic architecture is still useful when the project is small, the team is small, requirements are not stable, and the main goal is to validate the idea quickly. For a teaching demo, it is useful as a starting point for explaining why microservices and DDD become important.

1.2 Microservices Architecture

1.2.1 Concept

Microservices Architecture decomposes a system into small, independently deployable services. Each service owns one business capability and communicates with other services through explicit APIs such as REST or gRPC.

1.2.2 Characteristics

- Each service has its own codebase and deployment unit.
- Each service owns its data and avoids direct access to another service's database.
- Services communicate through APIs.
- Services can be developed, tested, scaled, and deployed independently.
- Failures can be isolated more clearly than in a monolith.

1.2.3 Monolithic vs Microservices

Table 1: Comparison between monolithic and microservices architecture.

Criteria	**Monolithic**	**Microservices**
Deployment	One application package	Many independently deployed services
Database	Usually one shared database	Database-per-service principle
Scaling	Scale the whole application	Scale each service independently
Development	Simple early development	Better for multiple teams and large domains
Failure impact	One module can affect the whole runtime	Failure can be isolated by service boundary
Complexity	Lower operational complexity	Higher distributed-system complexity

1.2.4 Advantages

- Independent scaling and deployment.
- Clearer business ownership.
- Easier technology evolution per service.
- Better fault isolation.
- Better fit for large systems and team-based development.

1.2.5 Disadvantages

- Network calls add latency and failure cases.
- Data consistency becomes harder.
- Deployment and monitoring are more complex.
- Debugging distributed workflows requires better logs and tracing.
- Service boundaries must be designed carefully.

1.2.6 Design Principles

Good microservice design follows high cohesion, loose coupling, database ownership, API contracts, automation, observability, and fault-tolerant communication. A service should represent a business capability rather than a technical layer.

1.3 Domain Driven Design (DDD)

1.3.1 Goal

Domain Driven Design helps model software around the business domain. Instead of starting from database tables or technical folders, DDD starts with domain language, business rules, bounded contexts, aggregates, entities, and value objects.

1.3.2 Core Concepts

- **Domain**: the business problem area.
- **Subdomain**: a smaller business area inside the domain.
- **Bounded Context**: a boundary where a model has a precise meaning.
- **Entity**: an object with identity, such as User, Product, or Order.
- **Value Object**: an object defined by value, such as Money or Address.
- **Aggregate**: a consistency boundary around related entities.
- **Context Map**: a diagram showing relationships between bounded contexts.

1.3.3 Context Map

In e-commerce, the word "product" can have different meanings in different contexts. In the Product Context, it is a live catalog item. In the Order Context, it becomes a snapshot attached to an order. In the AI Context, it can become a searchable recommendation document. These meanings should not be mixed in one model.

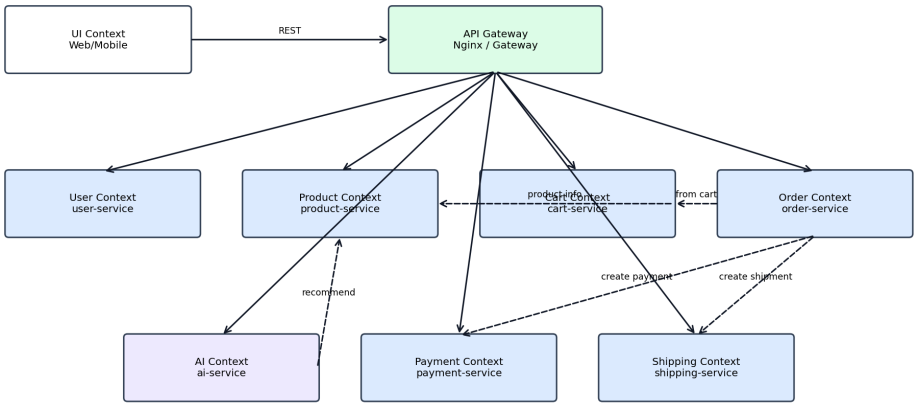

Figure 1: DDD context map for the target e-commerce system.

1.3.4 DDD in Microservices

DDD is useful for microservices because one bounded context is often a good candidate for one microservice. This does not mean every class becomes a service. The goal is to group behavior and data that change together.

1.4 Case Study: Healthcare (Practice Decomposition)

1.4.1 Problem Description

Healthcare is used here only as a decomposition case study, not as the main project of the essay. The purpose is to practice DDD thinking in a domain that has multiple business capabilities, such as patient identity, appointment, medical record, billing, pharmacy, and notification.

1.4.2 Step 1: Identify the Domain

The domain is a healthcare service system. The core business goal is to help users receive healthcare services safely and traceably. The system may contain patient registration, appointment booking, medical record management, pharmacy support, billing, and notification.

1.4.3 Step 2: Identify Bounded Contexts

The healthcare domain can be decomposed into these bounded contexts:

- Patient/User Context.
- Appointment Context.
- Medical Record Context.
- Pharmacy Context.
- Billing Context.
- Notification Context.

1.4.4 Step 3: Decompose into Microservices

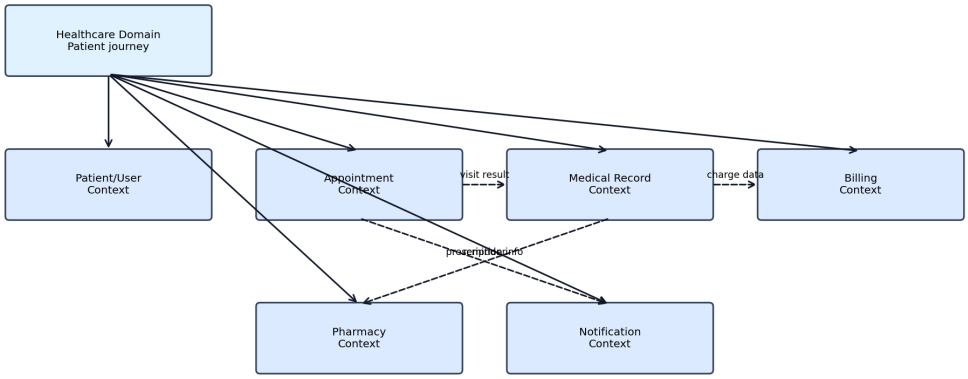

Figure 2: Healthcare case study decomposition for DDD practice.

Each context can become a service if it has enough independent business behavior. For example, appointment scheduling and billing should not share one database model because their business rules change for different reasons.

1.4.5 Step 4: Identify Relationships

The Appointment Context may call Notification to send reminders. The Medical Record Context may produce billing information. The Pharmacy Context may consume prescription-related information. These interactions should happen through APIs, not through direct database access.

1.4.6 Example API

Listing 1: Example healthcare API decomposition.

```
POST /api/patients/
POST /api/appointments/
GET  /api/medical-records/{patient_id}/
POST /api/pharmacy/dispense-requests/
POST /api/billing/invoices/
```

1.5 Conclusion

Monolithic architecture is suitable for simple early systems, but microservices are more appropriate when the system grows and the domain has clear boundaries. DDD provides a systematic way to identify those boundaries before creating services.

2 Developing an E-Commerce Microservices System

2.1 Requirement Identification

2.1.1 Functional Requirements

The main project is a general e-commerce system. It should support:

- Product management across multiple product domains such as books, electronics, and fashion.
- User management for admin, staff, and customer roles.
- Cart management.
- Order creation and order history.
- Payment processing or simulated payment.
- Shipping and delivery tracking.
- Product search and recommendation.
- AI-assisted product consultation through recommendation list and chatbot.

2.1.2 Non-functional Requirements

- **Scalability**: services can be scaled independently.
- **Security**: JWT authentication, role-based access control, and gateway validation.
- **Maintainability**: business logic is separated by bounded context.
- **Availability**: service failures should be isolated where possible.
- **Deployability**: Docker Compose should run the system locally.

2.2 DDD-Based System Decomposition

2.2.1 Bounded Contexts

Table 2: E-commerce bounded contexts and services.

Bounded context	Service	Responsibility
User Context	`user-service`	Authentication, authorization, profiles, roles
Product Context	`product-service`	Categories, products, inventory, search
Cart Context	`cart-service`	Cart, cart items, quantity updates

Bounded context	Service	Responsibility
Order Context	`order-service`	Order creation, order items, order lifecycle
Payment Context	`payment-service`	Payment records, payment status, confirmation
Shipping Context	`shipping-service`	Shipment creation and delivery status
AI Context	`ai-service`	Product recommendation and chatbot consultation

2.2.2 Principles

- Each context owns its own database.
- Services communicate through REST APIs.
- Frontend clients call through the API Gateway.
- No service should directly read or write another service's database.

2.3 Product Service Design (Django)

2.3.1 Product Classification

The product service supports multiple product domains:

- Book: textbooks, novels, reference books.
- Electronics: mobile phones, laptops, home appliances.
- Fashion: shirts, pants, shoes, accessories.

2.3.2 General Model

Listing 2: General product model.

```
class Category(models.Model):
    name = models.CharField(max_length=100)

class Product(models.Model):
    name = models.CharField(max_length=255)
    price = models.FloatField()
    stock = models.IntegerField()
    category = models.ForeignKey(Category, on_delete=models.CASCADE)
```

2.3.3 Domain-Specific Details

Listing 3: Domain-specific product extensions.

```
class Book(models.Model):
    product = models.OneToOneField(Product, on_delete=models.CASCADE)
    author = models.CharField(max_length=255)
    publisher = models.CharField(max_length=255)
    isbn = models.CharField(max_length=20)

class Electronics(models.Model):
    product = models.OneToOneField(Product, on_delete=models.CASCADE)
    brand = models.CharField(max_length=100)
    warranty = models.IntegerField()

class Fashion(models.Model):
    product = models.OneToOneField(Product, on_delete=models.CASCADE)
    size = models.CharField(max_length=10)
    color = models.CharField(max_length=50)
```

2.3.4 API

Listing 4: Product service API.

```
GET  /products/
POST /products/
GET  /products/{id}/
PUT  /products/{id}/
DELETE /products/{id}/
```

2.4 User Service Design (Django)

2.4.1 User Classification

The user service supports three roles:

- Admin: full system management.
- Staff: product, order, and shipping operations.
- Customer: product browsing, cart, checkout, and order history.

2.4.2 Model

Listing 5: User role model.

```
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    ROLE_CHOICES = (
        ("admin", "Admin"),
```

```
        ("staff", "Staff"),
        ("customer", "Customer"),
    )
    role = models.CharField(max_length=20, choices=ROLE_CHOICES)
```

2.4.3 Role-Based Access Control

Admin can manage the whole system. Staff can process orders, update products, and manage shipping. Customers can browse products, manage cart, place orders, and view their own order history.

2.4.4 API

Listing 6: User service API.

```
POST /auth/register/
POST /auth/login/
GET  /users/
GET  /users/me/
```

2.5 Cart Service Design

2.5.1 Model

Listing 7: Cart service model.

```
class Cart(models.Model):
    user_id = models.IntegerField()

class CartItem(models.Model):
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE)
    product_id = models.IntegerField()
    quantity = models.IntegerField()
```

2.5.2 Logic

The cart service handles adding products, updating quantity, removing items, and returning the current customer cart. It stores product identifiers but should not directly access the product database.

2.5.3 API

Listing 8: Cart service API.

```
POST   /cart/add/
GET    /cart/
PATCH  /cart/items/{id}/
DELETE /cart/remove/{id}/
```

2.6 Order Service Design

2.6.1 Model

Listing 9: Order service model.

```
class Order(models.Model):
    user_id = models.IntegerField()
    total_price = models.FloatField()
    status = models.CharField(max_length=50)

class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE)
    product_id = models.IntegerField()
    quantity = models.IntegerField()
```

2.6.2 Workflow

The order service creates an order from the cart, stores order items, calls the payment service, and then calls the shipping service. For a robust design, order items should store product snapshots so that historical orders do not change when product information changes.

2.7 Payment Service Design

2.7.1 Model

Listing 10: Payment service model.

```
class Payment(models.Model):
    order_id = models.IntegerField()
    amount = models.FloatField()
    status = models.CharField(max_length=50)
```

2.7.2 States

Payment status can be `pending`, `success`, or `failed`. A more detailed implementation may use `pending`, `paid`, `failed`, and `cancelled`.

2.7.3 API

Listing 11: Payment service API.

```
POST /payment/pay/
GET  /payment/status/{id}/
```

2.8 Shipping Service Design

2.8.1 Model

Listing 12: Shipping service model.

```
class Shipment(models.Model):
    order_id = models.IntegerField()
    address = models.TextField()
    status = models.CharField(max_length=50)
```

2.8.2 States

Shipping status can be `processing`, `shipping`, and `delivered`. A more complete design can include `pending`, `preparing`, `shipped`, `delivered`, and `cancelled`.

2.8.3 API

Listing 13: Shipping service API.

```
POST /shipping/create/
GET  /shipping/status/{id}/
PATCH /shipping/status/{id}/
```

2.9 Overall System Flow

1. User logs in through user-service.
2. User browses products through product-service.
3. User adds products to cart through cart-service.
4. User checks out and creates an order through order-service.
5. Order-service sends payment request to payment-service.
6. After payment succeeds, order-service creates shipment through shipping-service.

2.10 Practical Design Guide

2.10.1 Goal

The practical design part should include class diagrams, database mapping, and database design per service. This is required because the essay is not only theoretical; it must show how microservices are mapped into concrete design artifacts.

2.10.2 Class Diagram

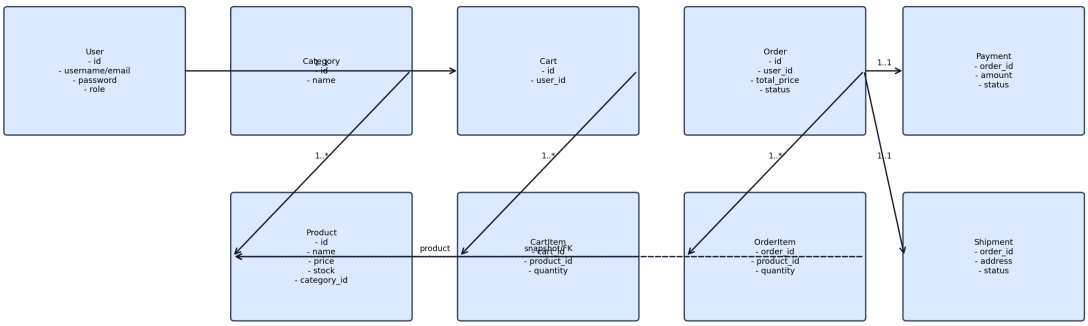

Figure 3: Class diagram for the target e-commerce microservices system.

2.10.3 Mapping Class Diagram to Database

The general mapping rule is:

- Class maps to table.
- Attribute maps to column.
- Relationship maps to foreign key or service-level identifier.
- Cross-service references should use IDs or snapshots, not direct database foreign keys.

2.10.4 Database Design for Each Service

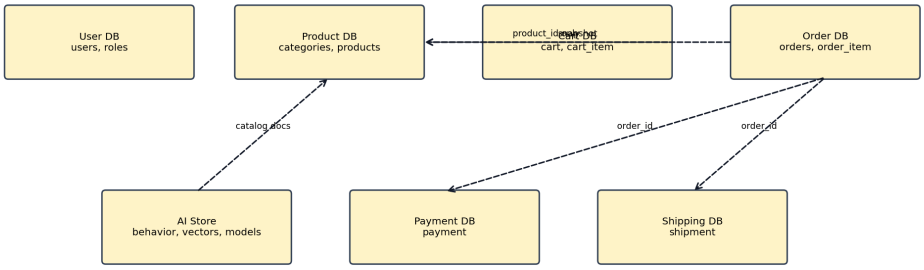

Figure 4: Database-per-service mapping for the target e-commerce system.

Table 3: Main database schema by service.

Service	**Suggested DB**	**Main tables**
Product Service	PostgreSQL	category, product, book, electronics, fashion
User Service	MySQL	user, role, profile
Cart Service	MySQL or PostgreSQL	cart, cart_item
Order Service	MySQL or PostgreSQL	orders, order_item
Payment Service	PostgreSQL	payment
Shipping Service	PostgreSQL	shipment
AI Service	PostgreSQL + Vector DB/Files	behavior events, embeddings, model artifacts

2.10.5 MySQL vs PostgreSQL

Table 4: MySQL and PostgreSQL comparison.

Criteria	**MySQL**	**PostgreSQL**
Popularity	Very common for web applications	Very common for complex applications
JSON support	Good	Strong
Complex queries	Good	Stronger for advanced queries
Authentication data	Suitable	Suitable
Catalog/search data	Suitable	Often preferred

2.10.6 Exercise

The final project evidence should export the class diagram from a diagram tool or include a clear generated diagram in the report. It should also show service-specific database schemas and explain why databases are not shared across services.

2.10.7 Evaluation Checklist

- Class diagram is clear and uses correct relationships.
- Database mapping is clear.
- Each service owns its own database.
- MySQL and PostgreSQL choices are explained.

2.11 Conclusion

The e-commerce system can be decomposed into user, product, cart, order, payment, shipping, and AI services. DDD helps ensure that each service maps to a business context rather than a random technical module.

3 AI Service for Product Consultation

3.1 Goal

The AI service improves the e-commerce customer experience by recommending products and answering product consultation questions. It can use user behavior, product similarity, and natural-language query context.

Expected outputs are:

- Recommended product list.
- Chatbot consultation response.

3.2 AI Service Architecture

The AI service is designed as an independent microservice. It receives user behavior and user queries, processes them through machine learning and retrieval components, and returns recommendations or chatbot responses.

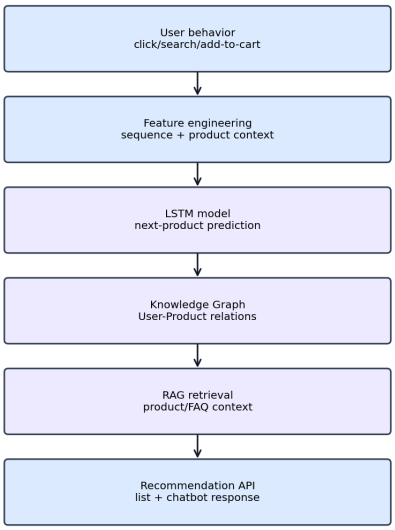

Figure 5: AI service pipeline for product consultation.

3.3 Data Collection

3.3.1 User Behavior Data

The service can collect:

- user_id
- product_id
- action: view, click, search, add_to_cart, buy
- timestamp
- query text or product category

3.3.2 Example Dataset

Listing 14: Example behavior dataset.

```
user_id,product_id,action,time
1,101,view,t1
1,102,add_to_cart,t2
2,205,buy,t3
```

3.4 LSTM Model (Sequence Modeling)

3.4.1 Idea

An LSTM model can predict the next product or category based on a sequence of user actions. For example, if a user views a phone, then views phone cases, then adds a charger to cart, the model may recommend related accessories.

3.4.2 Detailed Model

Listing 15: Simplified LSTM recommendation model.

```
import torch
import torch.nn as nn

class LSTMModel(nn.Module):
    def __init__(self, input_dim=10, hidden_dim=64, output_dim=100):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = out[:, -1, :]
        return self.fc(out)
```

3.4.3 Training

Listing 16: Simplified LSTM training loop.

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters())

for epoch in range(epochs):
    output = model(x)
    loss = criterion(output, y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
```

3.5 Knowledge Graph with Neo4j

3.5.1 Graph Model

A knowledge graph can represent relationships between users, products, categories, and behavior.

- Node types: User, Product, Category.
- Edge types: BUY, VIEW, ADD_TO_CART, SIMILAR, BELONGS_TO.

3.5.2 Cypher Example

Listing 17: Neo4j Cypher example.

```
CREATE (u:User {id: 1})
CREATE (p:Product {id: 101})
CREATE (u)-[:BUY]->(p)
```

3.5.3 Recommendation Query

Listing 18: Graph recommendation query.

```
MATCH (u:User {id: 1})-[:BUY]->(p)-[:SIMILAR]->(rec)
RETURN rec
```

3.6 RAG (Retrieval-Augmented Generation)

3.6.1 Pipeline

RAG combines retrieval and generation. The retrieval step searches product documents, FAQ content, and product descriptions. The generation step uses the retrieved context to answer the user's question.

3.6.2 Vector Database

The vector database can be FAISS or ChromaDB. Product descriptions are converted into embeddings and searched by semantic similarity.

3.6.3 Example

Listing 19: RAG search example.

```
query = "cheap gaming laptop"
results = vector_db.search(query)
response = llm.generate(query, context=results)
```

3.7 Hybrid Model

A hybrid recommender can combine:

- LSTM sequence score.
- Knowledge graph score.
- RAG semantic relevance score.

Listing 20: Hybrid recommendation score.

```
final_score = w1 * lstm_score + w2 * graph_score + w3 * rag_score
```

3.8 Two AI Service Forms

3.8.1 Recommendation List

Use cases:

- Product search page.
- Product detail page.

- Cart page.
- Home dashboard.

Listing 21: Recommendation API.

```
GET /recommend?user_id=1

Response:
[101, 102, 205]
```

3.8.2 Consultation Chatbot

The chatbot receives a natural-language question, detects intent, retrieves product candidates, and generates a response.

Listing 22: Chatbot API.

```
POST /chatbot

Input:
"I need a cheap laptop for study"

Output:
"You may consider these budget laptop products..."
```

3.9 AI Service Deployment

3.9.1 Tech Stack

- TensorFlow or PyTorch for LSTM.
- Neo4j for graph recommendation.
- FAISS or ChromaDB for vector retrieval.
- FastAPI or Django REST Framework for the AI service API.

3.9.2 Architecture

The AI service is independent and communicates with product and user behavior services through APIs. It should not directly modify product, cart, order, payment, or shipping databases.

3.10 Exercise

The AI exercise includes building a simple LSTM model, creating a product graph in Neo4j, implementing a recommendation API, and building a basic chatbot API.

3.11 Evaluation Checklist

- AI pipeline is clear.
- At least one model is implemented.
- Graph and RAG design are explained.
- AI API is available.
- Recommendation output can be tested.

3.12 Conclusion

AI improves e-commerce personalization by using behavior, product relationships, and language context. In a microservice system, AI should remain a separate service so that model dependencies do not pollute core commerce services.

4 Building the Complete System

4.1 Overall Architecture

4.1.1 System Model

The complete system is built using microservices. Each service is a Django project or Python service and owns its data.

- API Gateway (Nginx or dedicated gateway service).
- user-service.
- product-service.
- cart-service.
- order-service.
- payment-service.
- shipping-service.
- ai-service.

4.1.2 Principles

- Each service owns a separate database.
- Services communicate through REST APIs.
- Services do not directly access each other's databases.
- The gateway is the single entry point for external clients.

4.2 System Architecture

4.2.1 Overview

The proposed e-commerce system is a distributed microservice-based platform. It supports product browsing, cart, checkout, payment, shipping, and AI-assisted product consultation.

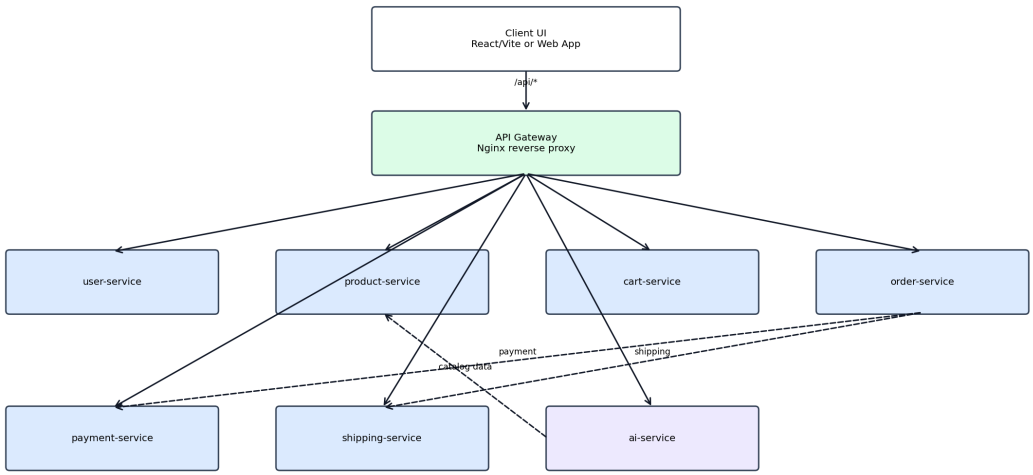

Figure 6: Microservice architecture of the target e-commerce system.

4.2.2 Microservice Architecture

Each business domain is implemented as an independent service. The services are independently deployable and maintain their own databases. This follows the database-per-service pattern.

4.2.3 API Gateway

The API Gateway is the single entry point. It routes client requests to the correct service, forwards authentication headers, performs basic validation, and can centralize logging and monitoring.

4.2.4 Service Communication

For the MVP, synchronous REST APIs are acceptable. Later versions can add asynchronous communication with Redis, RabbitMQ, Kafka, or another message broker.

4.2.5 Containerization and Deployment

All services are containerized with Docker. Docker Compose is used for local development and demonstration. Kubernetes can be added later for production-style orchestration.

4.2.6 System Structure

Listing 23: Suggested project structure.

```
ecom-final/
|-- gateway/
|   |-- nginx.conf
|-- user-service/
|-- product-service/
|-- cart-service/
|-- order-service/
|-- payment-service/
|-- shipping-service/
|-- ai-service/
|-- infrastructure/
|   |-- docker-compose.yml
```

4.2.7 Design Principles

The architecture follows loose coupling, high cohesion, independent scalability, fault isolation, explicit API contracts, and database ownership.

4.2.8 Security Considerations

Security is enforced through JWT authentication, role-based access control, gateway validation, and service-level authorization. Staff and customer capabilities must be separated both in the UI and backend APIs.

4.2.9 Discussion

Compared with a monolithic architecture, the microservice design improves flexibility and scalability but introduces distributed-system complexity. Docker, API Gateway routing, clear service contracts, and observability reduce this complexity.

4.3 API Gateway (Nginx)

4.3.1 Role

The gateway is responsible for:

- Acting as the entry point for the whole system.
- Routing requests to the correct service.
- Forwarding authentication headers.
- Logging API usage.
- Hiding internal service URLs from clients.

4.3.2 Sample Configuration

Listing 24: Sample Nginx gateway routing.

```
location /users/ {
    proxy_pass http://user-service:8000;
}

location /products/ {
    proxy_pass http://product-service:8001;
}

location /cart/ {
    proxy_pass http://cart-service:8002;
}
```

4.4 Authentication (JWT)

4.4.1 Installation

Listing 25: JWT dependency.

```
pip install djangorestframework-simplejwt
```

4.4.2 Configuration

Listing 26: JWT view configuration.

```
from rest_framework_simplejwt.views import TokenObtainPairView
```

4.4.3 Flow

1. User logs in and receives an access token.
2. Client sends the token in the Authorization header.
3. Gateway forwards the header.
4. Service validates the token and role.

4.5 Communication Between Services

4.5.1 REST API Call

Listing 27: REST call between services.

```
import requests

response = requests.get(
    "http://product-service:8001/products/",
    timeout=5
)
```

4.5.2 Best Practices

- Set request timeouts.
- Use retries carefully.
- Return clear errors.
- Use idempotency keys for payment and order operations.
- Add circuit breakers in advanced versions.

4.6 Dockerizing the System

4.6.1 Dockerfile for Django

Listing 28: Django Dockerfile.

```
FROM python:3.10
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

4.6.2 docker-compose.yml

Listing 29: Docker Compose sample.

```
version: "3"
services:
  user-service:
    build: ./user-service
    ports:
      - "8000:8000"

  product-service:
    build: ./product-service
    ports:
      - "8001:8001"
```

4.7 End-to-End System Flow

4.7.1 Use Case: Shopping

1. User logs in through user-service.
2. User views products through product-service.
3. User adds product to cart through cart-service.
4. User creates order through order-service.
5. Order-service calls payment-service.

6. Order-service calls shipping-service.

7. AI-service can recommend related products during browsing or checkout.

4.7.2 Sequence Logic

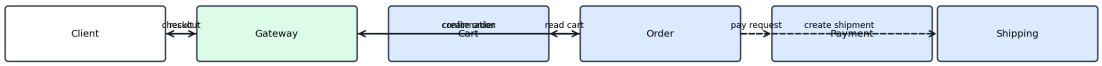

Figure 7: End-to-end checkout sequence for the e-commerce system.

4.8 Kubernetes Deployment (Optional)

4.8.1 Deployment

Listing 30: Kubernetes deployment example.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
```

4.8.2 Service

Listing 31: Kubernetes service example.

```
kind: Service
spec:
  type: ClusterIP
```

4.9 Logging and Monitoring

Logging can be implemented using the ELK stack. Monitoring can be implemented with Prometheus and Grafana. For a final production-style version, distributed tracing should be added to follow requests across gateway, order, payment, and shipping services.

4.10 System Evaluation

4.10.1 Performance

Performance can be evaluated by response time, throughput, database query time, and checkout latency.

4.10.2 Scalability

Each service can be scaled independently. Product-service can be scaled for browsing traffic, while payment-service can be protected more strictly.

4.10.3 Advantages

- Flexible service evolution.
- Clear service ownership.
- Better scalability.
- Better alignment with DDD.

4.10.4 Disadvantages

- More deployment complexity.
- More difficult debugging.
- More complex data consistency.
- Requires stronger monitoring and DevOps practice.

4.11 Practical Exercise

The practical part should implement Django services, connect them through APIs, dockerize the system, and test the full shopping flow with AI consultation output.

4.12 Evaluation Checklist

- API Gateway exists.
- JWT authentication exists.
- Docker can run the system.
- Order to payment to shipping flow works.
- AI recommendation or chatbot API works.
- Class diagram, data model, database mapping, and architecture diagrams are included.

Final Conclusion

This essay follows the lecturer's structure: it first explains monolithic architecture, microservices, and DDD; then it develops the main e-commerce microservice design; then it presents the AI service for product consultation; and finally it describes the complete system architecture, gateway, security, communication, Docker deployment, workflow, and evaluation.

The healthcare system appears only as the case study in Section 1.4 for practicing DDD decomposition. The main project remains the e-commerce microservices system with AI recommendation and chatbot consultation.

References

[1] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

[2] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.

[3] M. Fowler and J. Lewis, "Microservices: a definition of this new architectural term," 2014.

[4] C. Richardson, *Microservices Patterns*. Manning Publications, 2018.

[5] Django Software Foundation, "Django Documentation." Official documentation.

[6] Django REST Framework, "Django REST Framework Documentation." Official documentation.

[7] Neo4j, "Neo4j Graph Database Documentation." Official documentation.

[8] scikit-learn developers, "scikit-learn Documentation." Official documentation.

A Remaining Implementation Evidence to Add Later

Because the general e-commerce project is stored outside the current workspace, the final pass should replace generic design examples with code evidence from that project:

- Real repository folder structure.
- Actual service names and ports.
- Actual models and serializers.
- Actual API Gateway configuration.
- Actual Docker Compose file.
- Actual database engines and schema.
- Screenshots or command output proving the end-to-end flow.